

1. Os alunos dividiram-se em dois grupos e cada grupo escreveu uma lista de identificadores de localizações. No entanto, as duas listas ficaram completamente desordenadas. Para as reconciliar, pretendem ordenar ambas as listas e emparelhar o menor identificador da lista da esquerda com o menor da lista da direita, o segundo menor com o segundo menor, e assim sucessivamente. Por fim, devem calcular a soma das diferenças absolutas entre cada par de identificadores emparelhados.

Exemplo:

Se receberes as seguintes listas de IDs de localização:

3	4
4	3
2	5

A ordem deveria ser:

2	3
3	4
4	5

E a soma das diferenças absolutas é:

$$|(2-3)| + |(3-4)| + |(4-5)| = 1 + 1 + 1 = 3$$

Considera o teu input:

Lista esquerda:

{4239,54,1305,1929,2253,4189,166,4356,4304,4860,898,592,1720,1116,1580,3867,4352,1939,3698,4901,811,1616,1891,3671,1728,1671,1661,2045,3120,240,2158,4453,4627,350,3711,563,4594,2533,1727,2827,3795,4531,2442,1516,6,3515,1924,2601,2097,3661,2103,4324,108,4936,2281,2015,3710,3216,1905,973}

Lista direita:

{2781,2720,791,1169,2865,827,4536,1728,2125,384,167,4949,476,1046,1576,3666,2467,520,2384,1991,210,3258,3211,3744,4518,718,1934,1825,767,3689,4818,3014,1825,2918,3028,3029,3034,1727,1240,3754,1540,3396,935,3083,3048,1865,2011,252,3064,2876,2237,2675,245,2710,415,3192,4710,4747,3037,1299}

As duas listas têm 60 elementos.

A resposta é (escreve **apenas** o valor numérico):

R: 18012

2. A secretaria da universidade precisa de publicar a classificação oficial da turma antes do final do semestre. Cada estudante tem um registo com 2 campos: o seu número de identificação e um registo de atividades - uma string de códigos de uma única letra pertencentes ao conjunto {'L','E','P','R','S'} (Lecture, Exam, Project, Reading, Seminar). O registo de atividades codifica a pontuação de envolvimento de cada estudante: o número de tipos de atividades distintas frequentadas, multiplicado pelo comprimento da maior sequência consecutiva da mesma atividade.

```
int engagement(char *activity) {  
    return distinctActivities(activity) * longestStreak(activity); }
```

Um aluno pode ser representado pela seguinte struct

```
struct Student {  
    int number;  
    char activity [100]; }
```

O ranking dos alunos é determinado de acordo com os seguintes critérios: * Ordenar por pontuação de envolvimento de forma decrescente (maior pontuação primeiro). * Em caso de empate na pontuação, ordenar pelo número do aluno de forma crescente (menor número primeiro).

Exemplo:

```
int turma [] = {  
    { number: 8, activity: "EPPRRPP" },  
    { number: 5, activity: "LLLSSLL" },  
    { number: 24, activity: "RRRRRRRR" }  };
```

- O aluno 8 tem pontuação: $3 * 2 = 6$
- O aluno 5 tem pontuação: $2 * 3 = 6$
- O aluno 24 tem pontuação: $1 * 7 = 7$

Como os alunos 5 e 8 têm a mesma pontuação, o critério de desempate é o número do aluno. Assim, o aluno 5 fica à frente do aluno 8

Ordem final:

- * Rank 1 → number 24
- * Rank 2 → number 5
- * Rank 3 → number 8

Considera a seguinte turma com 27 estudantes:

```
struct Student escola [] = {  
    {5, "EPPRRPP"}, {8, "LLLSSLL"}, {24, "RPPRRRL"}, {3,  
    "SSSPSS"}, {1, "PPPEEEPPP"}, {15, "LRLLLLR"}, {11, "ERRREEE"}, {2,  
    "SSSLLLSS"}, {7, "PPLLPLL"}, {12, "RRSSSRR"}, {19,  
    "EEERRREEE"}, {4, "LLPPSSL"}, {9, "SSSEEESSL"}, {16,  
    "PPRRRLLL"}, {21, "LLLSSLLL"}, {6, "ERRRRPPP"}, {13,  
    "PPPSSSP"}, {25, "RRRLLLEERR"}, {10, "SSSPSS"}, {18, "LLLEEEELL"},  
    {14, "EEERRREEE"}, {17, "SSSPSSS"}, {20, "LLLEERRR"}, {22,  
    "PPLL LLLL"}, {23, "SSSRRSSS"}, {26, "EEPPPLL"}, {27,  
    "LLRRRRPPP" } };
```

Indica o número do estudante que se encontra na posição 15 do ranking (considera que o ranking começa em 1).

A resposta é (escreve apenas o valor numérico):

R: 8